

Week3_Day-4_Hands_On—Vasundhara

Problem 1 (Level-1): Student Feedback Form Interaction

Scenario:

A training institute wants to collect feedback from students through a simple web form. They want to enhance user experience using jQuery to validate inputs and display messages dynamically without reloading the page.

Requirements

- Create a feedback form with fields: Name, Email, Course Rating (dropdown), and Comments.
- Use jQuery selectors to capture form inputs.
- On clicking the Submit button, validate that Name and Email are not empty.
- Display a success message dynamically below the form using jQuery DOM manipulation.
- Clear the form fields after successful submission.

☐ Technical Constraints

- Use jQuery CDN for integration.
- Use basic jQuery syntax: `$(selector).action()`.
- Do not use plain JavaScript for validation (only jQuery methods).
- No backend integration required.

Expectations:

- Proper use of jQuery selectors and event handling.
- Clean and readable code structure.
- Validation messages should be user-friendly.

Learning Outcome:

- Understand jQuery syntax and selectors.
- Learn basic DOM manipulation using jQuery.

Source code:

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>Student Feedback Form</title>
6
7    <!-- jQuery CDN -->
8    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
9
10   <style>
11     body {
12       background-color: rgb(172, 203, 240);
13       font-family: Arial, sans-serif;
14       padding: 30px;
15     }
16     form {
17       width: 350px;
18     }
19     input, select, textarea, button {
20       width: 100%;
21       margin: 8px 0;
22       padding: 8px;
23     }
24     .error {
25       color: red;
26     }
27     .success {
28       color: green;
29       font-weight: bold;

```

```

27     .success {
28       color: green;
29       font-weight: bold;
30       margin-top: 10px;
31     }
32   </style>
33 </head>
34 <body>
35
36 <h2>Student Feedback Form</h2>
37
38 <form id="feedbackForm">
39   <input type="text" id="name" placeholder="Enter your Name">
40   <input type="email" id="email" placeholder="Enter your Email">
41
42   <select id="rating">
43     <option value="">Select Course Rating</option>
44     <option>Excellent</option>
45     <option>Good</option>
46     <option>Average</option>
47     <option>Poor</option>
48   </select>
49
50   <textarea id="comments" placeholder="Your comments"></textarea>
51
52   <button type="submit">Submit Feedback</button>

```

```

52     <button type="submit">Submit Feedback</button>
53 </form>
54
55 <div id="message"></div>
56
57 <script>
58 $(document).ready(function () {
59
60     $("#feedbackForm").submit(function (e) {
61         e.preventDefault();
62
63         let name = $("#name").val().trim();
64         let email = $("#email").val().trim();
65
66         $("#message").html("");
67
68         if (name === "" && email === "") {
69             $("#message").html("<p class='error'>Please enter both Name and Email.</p>");
70         } else if (name === "") {
71             $("#message").html("<p class='error'>Please enter your Name.</p>");
72         } else if (email === "") {
73             $("#message").html("<p class='error'>Please enter your Email.</p>");
74         } else {
75             $("#message").html("<p class='success'>Thank you! Your feedback has been submit
76             $("#feedbackForm")[0].reset();
77         }

```

```

73         $("#message").html("<p class='error'>Please enter your Email.</p>");
74     } else {
75         $("#message").html("<p class='success'>Thank you! Your feedback has been submitted.</p>");
76         $("#feedbackForm")[0].reset();
77     }
78 });
79
80 });
81 </script>
82
83 </body>
84 </html>

```

Output:

The image displays two screenshots of a web form titled "Student Feedback Form". The form contains four input fields: "Enter your Name", "Enter your Email", "Select Course Rating" (a dropdown menu), and "Your comments" (a text area). A "Submit Feedback" button is located at the bottom of the form.

The top screenshot shows the form after a submission attempt. The "Name" and "Email" fields are highlighted with a red border, and a red error message "Please enter both Name and Email." is displayed below the form.

The bottom screenshot shows the form after a successful submission. The "Name" and "Email" fields are no longer highlighted, and a green success message "Thank you! Your feedback has been submitted successfully." is displayed below the form.

Explanation:

1. Used `$(document).ready()` and `$("#feedbackForm").submit()` for event handling without page reload.
2. Used jQuery selectors with `.val()` and `.text()` for input capture and dynamic DOM updates.
3. Implemented validation and UI updates using `.addClass()`, `.removeClass()`, and `.reset()` methods.

Problem 2 (Level-1): Dynamic FAQ Toggle Section

Scenario:

A website contains a Frequently Asked Questions (FAQ) section. The company wants answers to appear and disappear when users click on the questions.

Requirements

- Create at least 5 FAQ questions with hidden answers.
- Use jQuery click() event to toggle answers.
- Apply slideToggle() or show()/hide() methods.
- Highlight the selected question using CSS class manipulation.

☐ Technical Constraints

- Use only jQuery methods for DOM manipulation.
- Use class selectors and ID selectors appropriately.
- Avoid inline JavaScript.

Expectations:

- Smooth UI interaction.
- Proper use of jQuery effects.
- Organized HTML structure.

Learning Outcome

- Implement event handling using jQuery.
- Use basic jQuery effects.
- Practice DOM traversal and manipulation.

Source code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>FAQ Section</title>
6
7   <!-- jQuery CDN -->
8   <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
9
10  <style>
11    body {
12      font-family: Arial, sans-serif;
13      background-color: #918a8a;
14      padding: 30px;
15    }
16
17    .faq-container {
18      width: 600px;
19      margin: auto;
20      background-color: white;
21      padding: 20px;
22      border-radius: 10px;
23      box-shadow: 0 0 10px rgba(0,0,0,0.1);
24    }
25
26    .faq-question {
27      padding: 15px;
28      margin: 5px 0;
29      background-color: #dba9a1;
```

```

30      background-color: #dba9a1;
31      cursor: pointer;
32      border-radius: 5px;
33      transition: background-color 0.3s;
34    }
35
36    .faq-question.active {
37      background-color: #72d1f9;
38      color: white;
39    }
40
41    .faq-answer {
42      display: none;
43      padding: 10px 15px;
44      margin-bottom: 10px;
45      background-color: #f9f9f9;
46      border-left: 3px solid #f2a363;
47      border-radius: 0 5px 5px 0;
48    }
49  </style>
50 </head>
51 <body>
52 <h2 style="text-align:center;">Frequently Asked Questions</h2>
```

```

50 <body>
51
52 <h2 style="text-align:center;">Frequently Asked Questions</h2>
53
54 <div class="faq-container">
55   <div class="faq-item">
56     <div class="faq-question">1. What is your return policy?</div>
57     <div class="faq-answer">Our return policy lasts 30 days...</div>
58   </div>
59   <div class="faq-item">
60     <div class="faq-question">2. How do I track my order?</div>
61     <div class="faq-answer">You can track your order using the tracking number sent to :
62   </div>
63   <div class="faq-item">
64     <div class="faq-question">3. Do you ship internationally?</div>
65     <div class="faq-answer">Yes, we ship to selected countries. Shipping fees apply...<
66   </div>
67   <div class="faq-item">
68     <div class="faq-question">4. How can I contact customer support?</div>
69     <div class="faq-answer">You can contact us via email at support@example.com or call
70   </div>
71   <div class="faq-item">
72     <div class="faq-question">5. Can I change my order after placing it?</div>
73     <div class="faq-answer">You can change your order within 2 hours of placing it...</
74   </div>
75 </div>
76

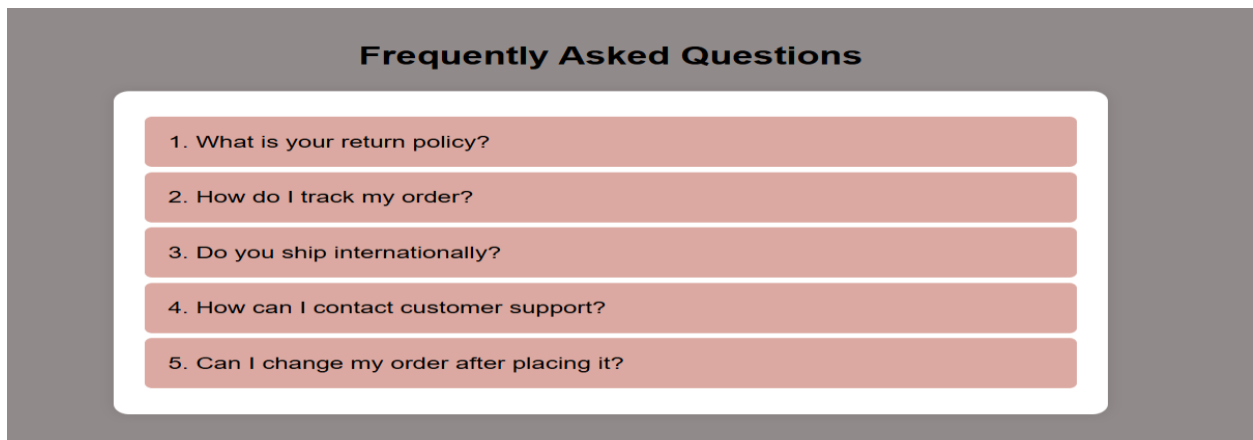
```

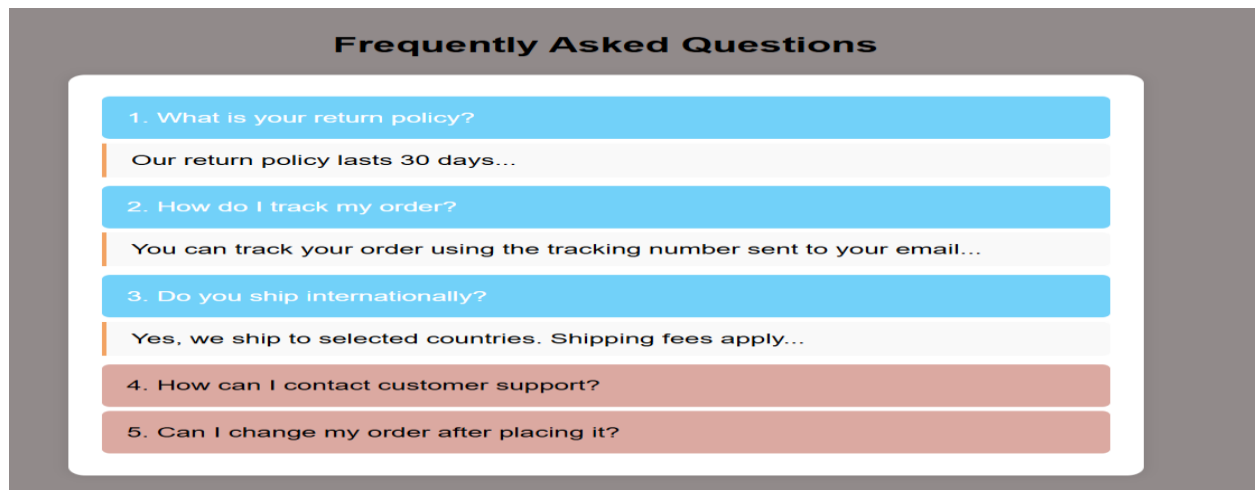
```

54 <div class="faq-container">
71   <div class="faq-item">
72     <div class="faq-question">5. Can I change my order after placing it?</div>
73     <div class="faq-answer">You can change your order within 2 hours of placing it...
74   </div>
75 </div>
76
77 <script>
78   $(document).ready(function() {
79     $(".faq-question").click(function() {
80       // Toggle current answer
81       $(this).next(".faq-answer").slideToggle(300);
82
83       // Highlight the clicked question
84       $(this).toggleClass("active");
85     });
86   });
87 </script>
88
89 </body>
90 </html>

```

Output:





Explanation:

1. Used jQuery `click()` event on FAQ question elements to dynamically toggle corresponding answers using `slideToggle()` for smooth UI interaction.
2. Applied class manipulation using `addClass()`, `removeClass()` or `toggleClass()` to highlight the selected question for better user experience.
3. Implemented structured HTML with proper class/ID selectors and handled all DOM manipulation purely through jQuery methods without inline JavaScript.

Problem 3 (Level-2): Product List with Add to Cart Counter

Scenario:

An e-commerce website wants to display a list of products. When users click 'Add to Cart', the cart counter at the top should increase dynamically.

Requirements

- Display at least 4 products with 'Add to Cart' buttons.
- Use jQuery to handle button click events.
- Increase the cart counter dynamically without page refresh.

- Disable the button once the product is added.
- Display a small confirmation message beside the product.

□ Technical Constraints

- Use jQuery event delegation if needed.
- Use .text(), .attr(), and .prop() methods.
- No backend or database integration.

Expectations:

- Efficient event handling.
- Proper use of jQuery methods for attribute manipulation.
- Clean UI updates.

Learning Outcome

- Understand advanced DOM manipulation.
- Use jQuery methods for dynamic content updates.
- Apply event delegation concepts.

source code:

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Product List with Cart Counter</title>
7      <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
8      <style>
9          body {
10             font-family: Arial, sans-serif;
11             margin: 20px;
12         }
13         .cart-counter {
14             font-size: 20px;
15             font-weight: bold;
16             margin-bottom: 20px;
17         }
18         .product {
19             border: 1px solid #ef4370;
20             padding: 15px;
21             margin-bottom: 10px;
22             position: relative;
23             width: 250px;
24         }
25         .confirmation {
26             color: green;
27             font-size: 14px;
28             margin-left: 10px;
29             display: none;

```

```

30     }
31     .product button:disabled {
32         background-color: #7d9fef;
33         cursor: not-allowed;
34     }
35 </style>
36 </head>
37 <body>
38
39 <div class="cart-counter">
40     Cart: <span id="cartCount">0</span>
41 </div>
42
43 <div class="products">
44     <div class="product" data-id="1">
45         <h3>Product 1</h3>
46         <p>Price: $10</p>
47         <button class="add-to-cart">Add to Cart</button>
48         <span class="confirmation">Added!</span>
49     </div>
50     <div class="product" data-id="2">
51         <h3>Product 2</h3>
52         <p>Price: $20</p>
53         <button class="add-to-cart">Add to Cart</button>
54         <span class="confirmation">Added!</span>

```

```

55     </div>
56     <div class="product" data-id="3">
57         <h3>Product 3</h3>
58         <p>Price: $15</p>
59         <button class="add-to-cart">Add to Cart</button>
60         <span class="confirmation">Added!</span>
61     </div>
62     <div class="product" data-id="4">
63         <h3>Product 4</h3>
64         <p>Price: $25</p>
65         <button class="add-to-cart">Add to Cart</button>
66         <span class="confirmation">Added!</span>
67     </div>
68 </div>
69
70 <script>
71 $(document).ready(function() {
72     // Event delegation for dynamic handling
73     $(".products").on("click", ".add-to-cart", function() {
74         // Get current cart count
75         let count = parseInt($("#cartCount").text());
76         // Increment cart count
77         $("#cartCount").text(count + 1);
78
79         // Disable the clicked button

```

```

70 <script>
71 $(document).ready(function() {
72     // Event delegation for dynamic handling
73     $(".products").on("click", ".add-to-cart", function() {
74         // Get current cart count
75         let count = parseInt($("#cartCount").text());
76         // Increment cart count
77         $("#cartCount").text(count + 1);
78
79         // Disable the clicked button
80         $(this).prop("disabled", true);
81
82         // Show confirmation message
83         $(this).siblings(".confirmation").text("Added!").show();
84     });
85 });

```

Output:

Cart: 0

Product 1

Price: \$10

Add to Cart

Product 2

Price: \$20

Add to Cart

Product 3

Price: \$15

Add to Cart

Product 4

Price: \$25

Add to Cart

Cart: 3

Product 1

Price: \$10

Add to Cart

Added!

Product 2

Price: \$20

Add to Cart

Added!

Product 3

Price: \$15

Add to Cart

Added!

Explanation:

1. Attached jQuery `click()` event to FAQ question elements and used `slideToggle()` to dynamically show and hide answers with smooth animation.
2. Used `toggleClass()` to apply a highlight CSS class on the selected question for visual feedback.
3. Managed DOM manipulation entirely using jQuery selectors (class and ID) without inline JavaScript for clean and organized structure.

Problem 4 (Level-2): Interactive To-Do List Application

Scenario:

A productivity web app requires a dynamic To-Do List where users can add, mark as complete, and remove tasks using jQuery.

Requirements

- Provide an input field and Add button to insert tasks.
- Display tasks dynamically in a list.
- Allow users to mark tasks as completed (strike-through effect).
- Provide a delete button for each task.
- Display total task count and completed task count dynamically.

☐ Technical Constraints

- Use jQuery for all DOM manipulation and event handling.
- Use `append()`, `remove()`, `toggleClass()`, and `on()` methods.
- No external libraries except jQuery.

Expectations:

- Proper dynamic element handling.
- Efficient event binding for dynamically created elements.
- Responsive and interactive UI behavior.

Learning Outcome

- Master jQuery DOM manipulation.
- Implement dynamic event handling.
- Build a small interactive application using jQuery

Source code:

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1.0">
6  <title>Interactive To-Do List</title>
7
8  <!-- jQuery CDN -->
9  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
10
11 <style>
12   body {
13     font-family: Arial, sans-serif;
14     background-color: #9ef558;
15     display: flex;
16     justify-content: center;
17     margin-top: 50px;
18   }
19
20   .container {
21     background: rgb(239, 100, 100);
22     padding: 20px;
23     width: 350px;
24     border-radius: 8px;
25     box-shadow: 0 0 10px rgba(0,0,0,0.1);
26   }
27
28   h2 {
29     text-align: center;
```

```

28     h2 {
29     }
30
31     input {
32         width: 70%;
33         padding: 8px;
34     }
35
36     button {
37         padding: 8px 10px;
38         cursor: pointer;
39     }
40
41     ul {
42         list-style-type: none;
43         padding: 0;
44         margin-top: 15px;
45     }
46
47     li {
48         padding: 8px;
49         margin-bottom: 5px;
50         background: #ea60c5;
51         display: flex;
52         justify-content: space-between;
53         align-items: center;
54

```

```

48         border-radius: 4px;
49     }
50
51     .completed {
52         text-decoration: line-through;
53         color: gray;
54     }
55
56     .counts {
57         margin-top: 10px;
58         font-size: 14px;
59     }
60
61     .delete-btn {
62         background: red;
63         color: white;
64         border: none;
65         padding: 5px 8px;
66         border-radius: 3px;
67     }
68 }
69 </style>
70 </head>
71
72 <body>
73
74 <div class="container">

```

```

80 <div class="container">
81   <h2>To-Do List</h2>
82
83   <!-- Input and Add Button -->
84   <input type="text" id="taskInput" placeholder="Enter a task">
85   <button id="addTask">Add</button>
86
87   <!-- Task List -->
88   <ul id="taskList"></ul>
89
90   <!-- Task Counts -->
91   <div class="counts">
92     Total Tasks: <span id="totalCount">0</span> |
93     Completed: <span id="completedCount">0</span>
94   </div>
95 </div>
96
97 <script>
98   $(document).ready(function(){
99
100     function updateCounts() {
101       let total = $("#taskList li").length;
102       let completed = $("#taskList li.completed").length;
103
104       $("#totalCount").text(total);
105       $("#completedCount").text(completed);
106     }

```

```

98   $(document).ready(function(){
100     function updateCounts() {
106     }
107
108     // Add Task
109     $("#addTask").on("click", function(){
110
111       let taskText = $("#taskInput").val().trim();
112
113       if(taskText !== "") {
114
115         let newTask = `
116           <li>
117             <span class="task-text">${taskText}</span>
118             <button class="delete-btn">Delete</button>
119           </li>
120         `;
121
122         // append() used here
123         $("#taskList").append(newTask);
124
125         $("#taskInput").val("");
126
127         updateCounts();
128       }
129     });

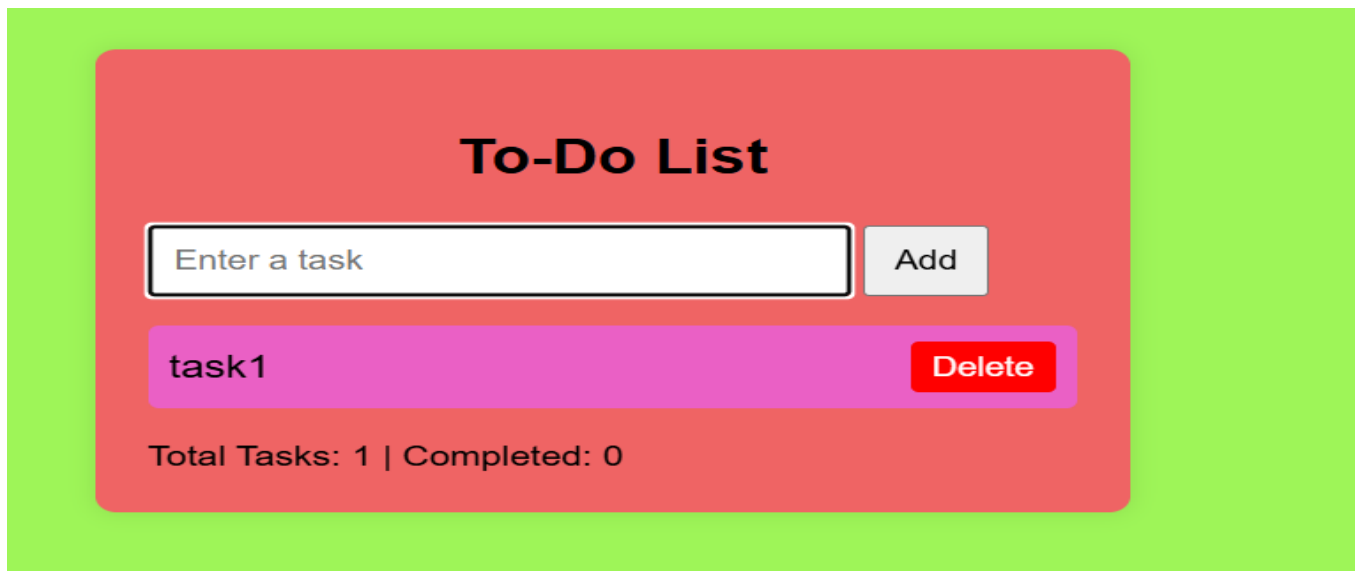
```

```

130
131     // Mark as Completed (Event Delegation using on())
132     $("#taskList").on("click", ".task-text", function(){
133
134         // toggleClass() used here
135         $(this).parent().toggleClass("completed");
136
137         updateCounts();
138     });
139
140     // Delete Task
141     $("#taskList").on("click", ".delete-btn", function(){
142
143         // remove() used here
144         $(this).parent().remove();
145
146         updateCounts();
147     });
148
149 });
150 </script>
151
152 </body>

```

Output:



The screenshot shows a web application titled "To-Do List" on a light green background. The application is contained within a red rounded rectangle. At the top, the title "To-Do List" is centered in bold black text. Below the title is a white input field with the placeholder text "Enter a task" and a grey "Add" button. Underneath the input field is a pink rounded rectangle containing a single task: "task1" in black text, followed by a red "Delete" button. At the bottom of the red container, the text "Total Tasks: 1 | Completed: 0" is displayed in black.

1. **Explanation:** Implemented task creation dynamically using `append()` to insert new `<li>` elements into the list and handled all interactions using jQuery `on()` for proper event delegation.
2. Used `toggleClass()` to apply strike-through styling for completed tasks and `remove()` to delete tasks dynamically while updating total and completed counts.
3. Managed all DOM manipulation and event handling purely with jQuery, ensuring efficient dynamic updates and responsive interactive UI behavior.